

文档密级：

课题名称：RISC-V 核心开发组件

RISC-V 核心开发组件

需求规格说明书

(版本：RuyiSDK 22.12)

编 制：陈小欧 陈小欧

审 核：邢明杰 邢明杰

批 准：邢明杰 邢明杰

完成日期：2022 年 11 月

中国科学院软件研究所

修订历史：

修订表					
版本号	修订描述	修订人	修订日期	审核人	审核日期
V1.0	新建文档	陈小欧	2022.11.30	邢明杰	2022.12.1

目 录

一、 简介	1
1.1 概述	1
1.2 术语与定义	4
1.3 需求目标	5
二、 功能需求综述.....	5
2.1 GCC 功能需求综述	5
2.2 LLVM 功能需求综述	9
2.3 V8 功能需求综述	15
2.4 OPENJDK 功能需求综述	17
2.5 QEMU 功能需求综述	23
2.6 OPENCV 功能需求综述	25
三、 详细功能需求.....	31
3.1 GCC 详细功能需求	31
3.2 LLVM 详细功能需求	37
3.3 V8 详细功能需求	43
3.4 OPENJDK 详细功能需求	45
3.5 QEMU 详细功能需求	51
3.6 OPENCV 详细功能需求	54
四、 非功能需求.....	61

4.1 GCC 非功能需求	61
4.2 LLVM 非功能需求	65
4.3 V8 非功能需求	69
4.4 OPENJDK 非功能需求	69
4.5 QEMU 非功能需求	69
4.6 OPENCV 非功能需求	73
五、 约束要求	75
5.1 GCC 约束要求	75
5.2 LLVM 约束要求	76
5.3 OPENCV 约束要求	78
六、 参考资料	78

一、简介

RISC-V 以其开源、开放的特性，为我国建立自主可控软硬件生态提供了重大机遇。基础软件则是整合 RISC-V 多样化生态的核心要素。当前，RISC-V 基础软件仍处于起步阶段，需要抓住机遇，加快 RISC-V 基础软件中核心开发组件的研发，抢占生态制高点，消除未来受制于人的隐患。

基础软件核心开发组件包括编译工具链、模拟器、开发环境等，是 RISC-V 基础软件生态的重要组成部分，是项目的核心研发内容。本课题旨在聚焦 RISC-V 基础软件核心开发组件的关键技术，在主流编程语言、编译工具链、开发环境、模拟器等方向形成突破，支撑 RISC-V 指令集上系统软件、并行软件、应用软件的开发。

基础软件核心开发组件的研发，将为 RISC-V 程序员提供针对 RISC-V 硬件平台的统一开发套件，相关研究成果将贡献到 RISC-V 上游开源社区，通过开放、共享的方式推动 RISC-V 软件生态建设，为国内 RISC-V 产业发展提供基础支撑。

1.1 概述

研究 RISC-V 基础软件中编译相关开发组件的关键技术，主要包括 RISC-V 平台上对主程序语言的支持、编译开发工具对 RISC-V 扩展指令集的支持和优化、以及在扩展指令集标准草案阶段用于早期测试验证的 RISC-V 模拟器支持。并将研究成果进行工程实

现，贡献到 RISC-V 上游开源社区，通过开放、共享的方式推动 RISC-V 软件生态，为国内 RISC-V 产业发展提供基础支撑。

课题的任务组织如下图所示：

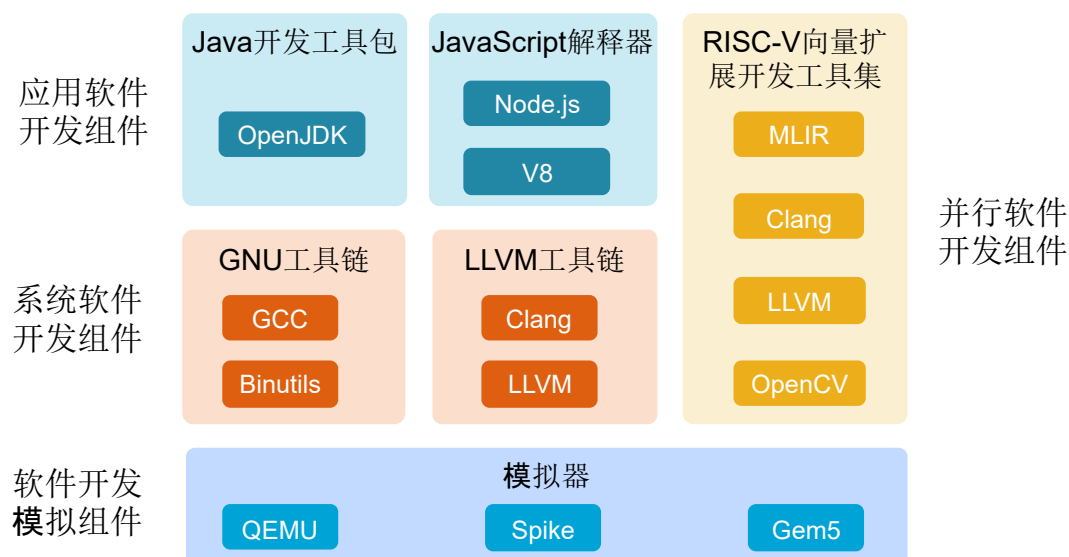


图 1 RISC-V 核心开发组件任务组织图

具体任务划分如下：

研究任务一：面向 RISC-V 系统软件的编译工具支持和优化

面向 RISC-V 底层系统软件开发领域，针对 C、C++ 编程语言提供 LLVM、GCC 编译工具链的支持和优化，研究和实现在工具链中对 RISC-V 扩展指令集架构的支持。

研究任务二：面向 RISC-V 应用软件的开发工具支持和优化

面向 RISC-V 上层应用软件开发领域，针对 Java、JavaScript 等编程语言提供相应开发工具包的支持和优化，研究和实现在开发工具包中对 RISC-V 扩展指令集架构的支持。

研究任务三：面向 RISC-V 并行软件的开发工具支持和优化

面向 RISC-V 并行软件开发，针对高性能计算、计算视觉、人工

智能等领域提供相应开发工具集的支持和优化，研究和实现在开发工具集中对 RISC-V 向量扩展指令集架构的支持，从而充分利用 RISC-V 向量扩展的硬件特性，提高程序性能；研究图形化思维导图到规划行为语言的解析和规约，探索其进一步向 C++ 等高级语言的映射和并行优化，初步形成面向专用领域的 RISC-V 指令集工具链原型系统。

RISC-V 向量扩展使用向量(vector)的英文 V 表示，它是 RISC-V 指令集的标准扩展模块之一。RISC-V 向量扩展最早在 2019 年 6 月 13 日发布了 0.7.1 版本提案，之后不断更新完善，先后发布了 0.8、0.9、0.10 版本，最终于 2022 年冻结形成 1.0 版本。截止目前，RISC-V 向量扩展规范仍处于 1.0 版本的公众评审阶段。

RISC-V 向量扩展是一种基于向量体系结构设计的用于数据并行加速的指令集，与大部分 SIMD 指令集不同，RISC-V 向量扩展可支持不同寄存器大小的硬件架构。常量参数 VLEN 代表具体的向量寄存器位数，由硬件架构决定，即 RISC-V 向量扩展的每个向量寄存器大小均为 VLEN。RISC-V 向量扩展在 RISC-V 基础模块上新增了 32 个向量寄存器及 600 多条向量指令。RISC-V 向量扩展的指令均采用 32 位编码，每条指令都支持相应的版本。同时 RISC-V 向量扩展也具有和当前流行 SIMD 指令集不同的特点。

研究任务四：面向 RISC-V 软件开发的模拟验证平台

面向 RISC-V 软件开发，基于 QEMU、Spike、Gem5 等模拟器提供相应指令集架构的模拟验证支持，研究和实现在模拟器中对

RISC-V 扩展指令集架构的支持。

由于目前 RISC-V 的指令集标准尚处于发展阶段，并且还有一些 RISC-V 扩展指令集处于草案阶段，需要不断的更新、演变。新定义的指令和机制需要首先通过模拟器进行实现，并提供验证环境。用户编写的应用程序也需要首先通过模拟器进行功能验证。在目前支持 RISC-V 的主流模拟器中，由于各个模拟器的维护力度不同，因此，对 RISC-V 各种扩展指令集的支持情况也参差不齐，亟需支持完善。

1.2 术语与定义

RISC-V 指令集架构：一种基于精简指令集计算（RISC）原则的开源指令集架构（ISA），主要特点是其开源性、可定制性和高度可扩展性；

RISC-V 指令集扩展：RISC-V 指令集架构由基础整数指令集和各种扩展指令集组成，扩展指令集分为标准扩展指令集和非标准扩展指令集；

RISC-V 核心开发组件：涵盖编译器、模拟器和运行时/虚拟机等；

编译器：主要负责将源代码编译、汇编和链接为目标指令集实现的二进制代码；

语言运行时/虚拟机：为解释型编程语言开发的软件提供开发环境，确保软件能够在 RISC-V 架构的设备上运行；

模拟器：为开发者提供仿真运行环境，使得开发者不受硬件设备的限制。

1.3 需求目标

本课题的目标是面向 RISC-V 开放指令集生态需求，研发 RISC-V 基础软件核心开发组件，在编程语言、编译工具链、开发环境、模拟器等方向形成技术突破，支撑 RISC-V 指令集上系统软件、并行软件、应用软件开发，为 RISC-V 程序员提供针对 RISC-V 硬件平台的统一开发套件。

二、 功能需求综述

2.1 GCC 功能需求综述

GNU 编译工具链（GNU Toolchain）是一套用于开发应用程序和操作系统的开源工具集合，广泛应用于 Linux、嵌入式系统及其他平台的软件开发。它主要由以下几个核心组件构成，每个组件在代码编译、链接、调试和优化过程中扮演关键角色：

(1) GNU Compiler Collection (GCC)

GCC 是 GNU 工具链的核心编译器，支持多种编程语言（如 C、C++、Fortran、Ada 等），能够将源代码编译为目标代码。其主要功能包括：多语言支持，通过不同的前端（如 `gcc` 用于 C，`g++` 用于 C++）处理不同语言的源代码。优化选项，提供 `-O1`、`-O2`、`-O3` 等优化级别，提升代码执行效率。交叉编译，支持生成不同架构（如 ARM、MIPS、RISC-V）的目标代码，适用于嵌入式开发。代码覆盖率与性能分析，集成 `gcov`（代码覆盖率）和 `gprof`（性能分

析) 工具。

(2) GNU Binutils

Binutils 是一组二进制工具，用于处理目标文件和可执行文件，主要包括：汇编器 (as)，将汇编代码转换为机器码。链接器 (ld)，合并多个目标文件，生成可执行文件或共享库。objdump，反汇编二进制文件。objcopy，转换目标文件格式。readelf，分析 ELF 文件结构。nm，列出符号表。

(3) GNU C Library (glibc)

glibc 是 GNU 的 C 标准库实现，提供系统调用、内存管理、文件操作等基础功能。在嵌入式开发中，也可选用轻量级替代品如 uClibc 或 newlib。

(4) GNU Debugger (GDB)

GDB 是强大的调试工具，支持断点设置、单步执行、变量查看等功能，适用于调试 C/C++ 程序。

(5) GNU Make

Make 用于自动化构建过程，通过 'Makefile' 定义编译规则，管理依赖关系。

(6) 其他辅助工具

- Autotools (Autoconf, Automake, Libtool)：用于跨平台构建配置。
- BFD (Binary File Descriptor) 库：提供统一的二进制文件操作接口，被 Binutils 和 GDB 使用。

RISC-V 核心开发组件中 GCC 对 RISC-V 架构的支持需求总结如

表 1 所示，本年度将实现汇编层（Binutils）对 B、K、P、Zce、Zfinx 扩展的支持，对应表 1 中 1-5 项需求。

表 1 GCC 的功能需求清单

序号	模块	子模块	需求	需求优先级	需求描述
1	GCC	Binutils	在 Binutils 中添加对 RISC-V B 扩展的汇编支持	高	在 Binutils 中添加对 RISC-V B 扩展的汇编支持，以确保能够正确识别、解析并生成对应的机器码。
2			在 Binutils 中添加对 RISC-V K 扩展的汇编支持	高	在 Binutils 中添加对 RISC-V K 扩展的汇编支持，以确保能够正确识别、解析并生成对应的机器码。
3			在 Binutils 中添加对 RISC-V P 扩展的汇编支持	高	在 Binutils 中添加对 RISC-V P 扩展的汇编支持，以确保能够正确识别、解析并生成对应的机器码。
4			在 Binutils 中添加对 RISC-V	高	在 Binutils 中添加对 RISC-V Zce 扩

			Zce 扩展的汇编支持		展的汇编支持，以确保能够正确识别、解析并生成对应的机器码。
5			在 Binutils 中添加对 RISC-V Zfinx 扩展的汇编支持	高	在 Binutils 中添加对 RISC-V Zfinx 扩展的汇编支持，以确保能够正确识别、解析并生成对应的机器码。
6		GCC	在 GCC 中添加对 RISC-V B 扩展的代码生成支持	中	在 GCC 中添加对 RISC-V B 扩展的代码生成支持，实现从中间表示到汇编码的转换。
7			在 GCC 中添加对 RISC-V K 扩展的代码生成支持	中	在 GCC 中添加对 RISC-V K 扩展的代码生成支持，实现从中间表示到汇编码的转换。
8			在 GCC 中添加对 RISC-V P 扩展的代码生成支持	中	在 GCC 中添加对 RISC-V P 扩展的代码生成支持，实现从中间表示到汇编码的转换。

9			在 GCC 中添加对 RISC-V Zce 扩展的代码生成支持	中	在 GCC 中添加对 RISC-V Zce 扩展的代码生成支持，实现从中间表示到汇编码的转换。
10			在 GCC 中添加对 RISC-V Zfinx 扩展的代码生成支持	中	在 GCC 中添加对 RISC-V Zfinx 扩展的代码生成支持，实现从中间表示到汇编码的转换。
11		GCC	根据指令集特点，优化编译性能	低	充分利用扩展指令集特性，挖掘优化机会，提高编译优化性能

2.2 LLVM 功能需求综述

LLVM (Low Level Virtual Machine) 是一个开源的编译器基础设施项目，旨在提供模块化、可重用的编译器与工具链技术。它采用现代化的设计理念，将前端（如 Clang）、优化器和后端分离，支持多种编程语言（如 C/C++、Rust、Swift）和硬件架构（如 x86、ARM、GPU）。LLVM 的核心是其高效的中间表示（IR），允许开发者进行跨平台的代码优化与生成。凭借其灵活性、高性能和活跃的社区，LLVM 已成为现代编译器开发（如 Clang、Rustc）和 JIT 编译（如 Julia）的核心引擎，广泛应用于操作系统、嵌入式系统和高

性能计算领域。LLVM 是一个模块化的编译器基础设施，主要由以下几个核心模块组成：

(1) 前端 (Frontend)

LLVM 的前端负责将高级编程语言（如 C/C++、Rust、Swift）的源代码解析并转换为 LLVM 中间表示（IR）。不同的语言可以使用不同的前端，例如 Clang（C/C++）、Rustc（Rust）和 SwiftC（Swift）。前端的主要任务是进行词法分析、语法分析、语义检查和 IR 生成，确保代码符合语言规范并优化初始 IR 结构。

(2) 中间表示 (LLVM IR)

LLVM IR 是 LLVM 的核心数据结构，它是一种与硬件无关的低级语言，采用静态单赋值（SSA）形式，便于优化和分析。IR 可以是人类可读的文本格式（.ll）或二进制格式（.bc），支持跨平台优化，并作为前端和后端之间的桥梁，使不同语言的代码能够在统一的框架下进行优化和代码生成。

(3) 优化器 (Optimizer)

LLVM 的优化器对 IR 进行各种优化，以提高代码的执行效率或减少体积。优化过程由多个 Pass（优化阶段）组成，例如死代码消除（DCE）、内联展开（Inlining）、循环优化（Loop Unrolling）等。开发者可以自定义优化策略，使 LLVM 适用于不同场景，如高性能计算或嵌入式系统。

(4) 后端 (Backend)

后端负责将优化后的 IR 转换为目标机器的机器码（如 x86、

ARM、RISC-V）。它包含多个关键步骤，如指令选择（将 IR 映射到机器指令）、寄存器分配（决定变量如何存储在寄存器中）和指令调度（优化指令执行顺序）。LLVM 支持多种架构，并可扩展以支持新的硬件平台。

(5) 运行时库（Runtime Libraries）

LLVM 提供了一系列运行时库，用于支持底层操作，如内存管理、异常处理和调试功能。例如，`compiler-rt` 提供编译器内置函数的实现（如 `memcpy`、`sqrt`），而 ASAN（AddressSanitizer）和 TSAN（ThreadSanitizer）用于检测内存错误和线程竞争问题。

(6) JIT 编译器（Just-In-Time Compilation）

LLVM 的 JIT 编译器能够在程序运行时动态编译 IR 并执行，适用于解释型语言（如 Julia、LuaJIT）和即时优化场景（如游戏脚本引擎）。JIT 结合了解释器的灵活性和静态编译的高效性，支持运行时优化和热代码替换。

(7) 链接器（LLD）

LLD 是 LLVM 原生的高性能链接器，支持 ELF（Linux）、COFF（Windows）和 Mach-O（macOS）等可执行文件格式。它比传统链接器（如 GNU ld）更快，并深度集成 LLVM 工具链，适用于大型项目的快速构建。

(8) 调试与工具链

LLVM 提供了一系列开发工具，包括 LLDB（现代化的调试器，类似 GDB）、libc++（LLVM 的 C++ 标准库实现）以及命令行工具

（如 `opt` 用于 IR 优化、`llc` 用于生成汇编代码）。这些工具增强了 LLVM 的调试、分析和代码生成能力，使其适用于完整的编译器开发生态。

LLVM 添加对 RISC-V 扩展指令集的支持主要涉及后端模块的扩展，这一过程需要在编译流程和汇编处理两个关键环节进行协同优化。在编译层面，LLVM 需要能够识别并处理 RISC-V 的各种扩展指令集，包括正确解析指令语义、优化寄存器分配策略，以及根据目标平台特性进行指令调度和代码优化。在汇编层面，系统需要支持扩展指令的汇编与反汇编功能，确保能够正确生成和解析包含扩展指令的目标代码，同时保持与标准工具链的兼容性。

RISC-V 核心开发组件中 LLVM 对 RISC-V 架构的支持需求总结如表 2 所示。本年度将实现汇编层对 B、K、P、Zce、Zfinx 扩展的支持，对应表 2 中 1-5 项需求。

表 2 LLVM 的功能需求清单

序号	模块	子模块	需求	需求优先级	需求描述
1	LLVM	汇编模块	在汇编模块中添加对 RISC-V B 扩展的汇编支持	高	在汇编模块中添加对 RISC-V B 扩展的汇编支持，以确保能够正确识别、解析并生成对应的机器码。
2			在汇编模块中	高	在汇编模块中添加

			添加对 RISC-V K 扩展的汇编支持		对 RISC-V K 扩展的汇编支持，以确保能够正确识别、解析并生成对应的机器码。
3			在汇编模块中添加对 RISC-V P 扩展的汇编支持	高	在汇编模块中添加对 RISC-V P 扩展的汇编支持，以确保能够正确识别、解析并生成对应的机器码。
4			在汇编模块中添加对 RISC-V Zce 扩展的汇编支持	高	在汇编模块中添加对 RISC-V Zce 扩展的汇编支持，以确保能够正确识别、解析并生成对应的机器码。
5			在汇编模块中添加对 RISC-V Zfinx 扩展的汇编支持	高	在汇编模块中添加对 RISC-V Zfinx 扩展的汇编支持，以确保能够正确识别、解析并生成对应的机器码。
6		代码生成模块	在代码生成模块中添加对	中	在代码生成模块中添加对 RISC-V B

			RISC-V B 扩展的代码生成支持		扩展的代码生成支持，实现从中间表示到汇编码的转换。
7			在代码生成模块中添加对 RISC-V K 扩展的代码生成支持	中	在代码生成模块 LLVM 中添加对 RISC-V K 扩展的代码生成支持，实现从中间表示到汇编码的转换。
8			在代码生成模块中添加对 RISC-V P 扩展的代码生成支持	中	在代码生成模块中添加对 RISC-V P 扩展的代码生成支持，实现从中间表示到汇编码的转换。
9			在代码生成模块中添加对 RISC-V Zce 扩展的代码生成支持	中	在代码生成模块中添加对 RISC-V Zce 扩展的代码生成支持，实现从中间表示到汇编码的转换。
10			在代码生成模块中添加对 RISC-V Zfinx 扩展的代码生成支持	中	在代码生成模块中添加对 RISC-V Zfinx 扩展的代码生成支持，实现从中间表示到汇编码的转换。

			扩展的代码生成支持		生成支持，实现从中间表示到汇编码的转换。
11		优化器/后端	根据指令集特点，优化编译性能	低	充分利用扩展指令集特性，挖掘优化机会，提高编译优化性能

2.3 V8 功能需求综述

表 3 V8 的功能需求清单

序号	模块	子模块	需求	需求优先级	需求描述
1	JavaScript 引擎 V8 对 RISC-V 的 64 位和 32 位基础指令集的支持	构建系统适配	新增构建系统中 RISC V64 和 RISC V32 配置	高	提供任务配置参数，完成构建任务的配置。
		RISC-V 64 位和 32 位基础指令集的功能支持	64 /32 位基础指令集（RV64I/M/F/D/A）功能支持	高	对 RISC V 通用指令集的支持

	功 能 支持	32/64 位运行时环境适配	实现常量池、重定位等功能	高	适配实现
		寄存器分配和内存管理	实现寄存器分配和内存管理 API	高	架构支持
		指令集兼容性与性能优化	实现 32/64 位指令集架构的代码复用	高	代码可读性和简洁性
2	JavaS-cript 引擎 V8 对 RISC-V 的 64 位和 32 位扩展指令集的功能支持	C 指令集的功能支持 (zca zcd zcf)	64/32 位压缩指令集功能支持	高	对 RISC-V 压缩指令集的支持
		Zfh (半精度浮点扩展)	对接指令选择和代码生成, 对 Wasm FP16 opcode 生成 Zfh 指令	中	对 Wasm FP16 支持
		Zbb/Zbs/Zba (位操作扩展)	对接指令选择和代码生成, 对 JavaScript 的位操作运算生成 Zbb/Zbs/Zba 指令	高	对 bit manipulation 支持
		Zicond (条件指令扩展)	对接指令选择和代码生成, 对 JavaScript 的条件赋值操作生成 Zicond 指令	高	对 Zicond 支持

		Zmmul（乘法扩展支持）	对接指令选择和代码生成，对 JavaScript 的乘法操作使用 Zmmul 指令集	中	对 Zmmul 支持
		向量扩展（RVV 支持）	对接指令选择和代码生成，对 WASM SIMD 操作码使用 RVV 指令集	高	对向量扩展支持

2.4 OpenJDK 功能需求综述

Java 是一门非常重要的主流编程语言，在 TIOBE 2021 年度全球编程语言排行榜中位于第 2 名，仅次于 C 语言。其具有面向对象、安全性、平台独立与可移植性、动态性等特点，适用于编写桌面应用程序、Web 应用程序、分布式系统和嵌入式系统应用程序。

OpenJDK（Open Java Development Kit）是 Java 平台标准版（Java SE）的官方开源实现，由 Sun Microsystems 于 2006 年启动，现由 Oracle 主导，并依托全球开发者社区共同维护。它是 Java 生态系统的核心，为 Java 应用程序提供运行环境（JVM）和开发工具（如编译器、调试器等）。同时，OpenJDK 是 Java 生态的基石，提供高性能 JVM、丰富的 API 和工具链。其开源模式促进了广泛采用，各大厂商提供优化版本，适用于从嵌入式到云计算的各类场景。

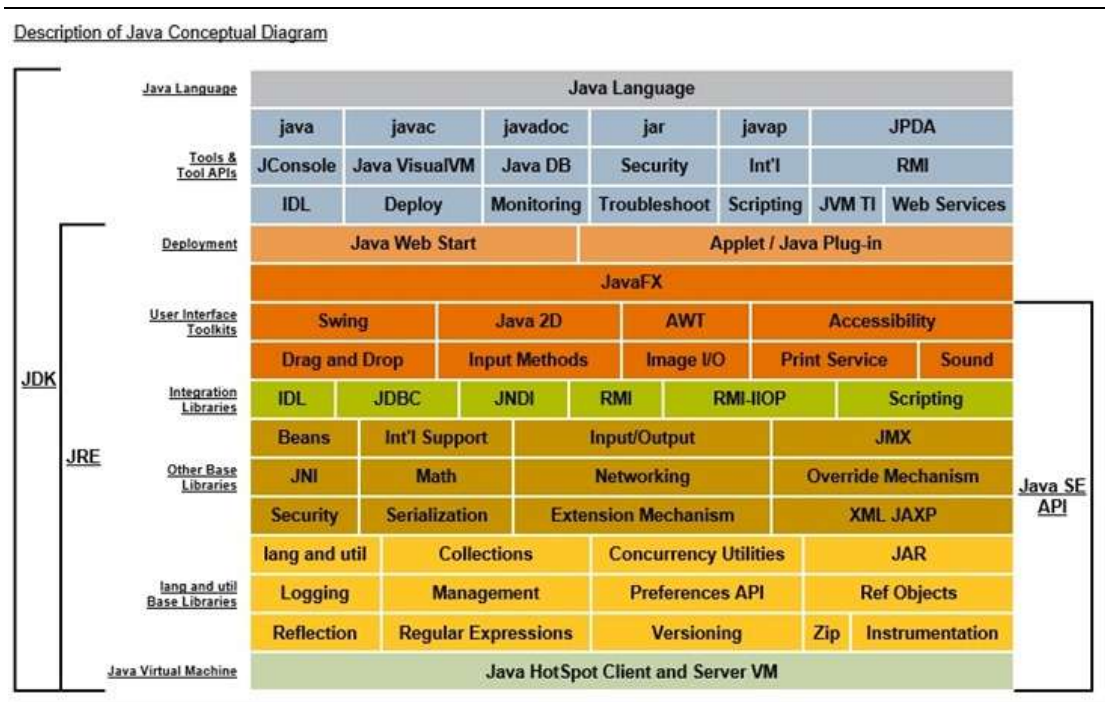


图 2 OpenJDK 整体框架图

OpenJDK 的主要功能包括：

(1) **Hotspot:** HotSpot 是 OpenJDK 的默认 Java 虚拟机（JVM），也是目前 Java 生态系统中性能最优、应用最广泛的 JVM 实现。HotSpot 采用混合执行模式，结合了解释执行和即时编译（JIT），并具备先进的垃圾回收机制。其核心架构包括解释器和即时编译器。其中，解释器负责逐行解释字节码，启动快单执行效率较低，适用于短声明周期方法或者冷代码。即时编译器包含 C1 和 C2，其中 C2 负责深度优化，是默认使用的即时编译器。

(2) **模板解释器:** 模板解释器是 HotSpot 解释器中的核心组成部件，是默认字节码的执行引擎，通过预生成平台相关的机器码模板（每个字节码对应一段汇编指令）提升性能，显著优于纯软件实现的解释器（如 Zero 解释器）。它性能优化的核心是通过将 JVM 字节码直接编译为平台相关的汇编指令片段，消除传统解释器的二次

解码开销，然后在 JVM 启动时根据 TemplateTable 动态组装完整解释器，实现高效指令调度。

(3) **Zero 解释器**：OpenJDK 的 Zero 解释器是一种纯 C++ 实现的解释器，主要用于提供跨平台支持，特别是在缺乏平台特定汇编优化的场景下工作。Zero 解释器的核心是跨平台设计，通过完全移除平台相关的汇编代码，使用纯 C++ 实现字节码解释执行，使 HotSpot JVM 能运行于未优化支持的硬件架构。相比依赖汇编的模板解释器（Template Interpreter），Zero 牺牲了执行效率以换取可移植性。

(4) **垃圾回收器**：OpenJDK 提供了多种垃圾回收器，每种设计针对不同场景优化，主要围绕吞吐量、延迟和内存占用三大指标进行权衡。主要包括：Serial GC、并行回收器、并发标记清除（CMS GC）、G1 回收器（Garbage-First）、ZGC（Z Garbage Collector）和 Shenandoah GC，这些回收器都有着各自的优点和缺点，都曾经或者正在使用在不同的发行版本中。

OpenJDK 除了上述主要的功能模块之外，还有线程与并发、监控与调试工具等。OpenJDK 的模板解释器和即时编译器都强依赖于指令集架构，需要针对专门的指令集架构进行移植；Zero 解释器不依赖于指令集架构，但是需要根据指令集做微调之后即可使用。RISC-V 作为一种指令集架构，也不例外，OpenJDK 的模板解释器和即时编译器都需要针对 RISC-V 做专门的移植，Zero 解释器只需要针对 RISC-V 做微调即可。

RISC-V 指令集针对目标架构的位数，可以分为 64 位架构（RV64）和 32 位架构（RV32）。RV64 和 RV32 在寄存器宽度、寻址空间、数据模型和指令长度，乃至指令本身的范围和数量都不相同。因此，OpenJDK 要面向 RISC-V，就需要面向 RV64 和 RV32 两种指令集，分别进行移植和适配。

RISC-V 是一种开源开放的精简指令集计算（RISC）架构，其设计核心在于模块化、简洁性和可扩展性。因此，RISC-V 指令集分为基础指令集和可选扩展指令集。基础指令集通常指的是整数指令集 I，通过将它与一些常用的扩展指令集进行组合，可以得到一个方便实用的指令集组合。其中，G 就是最常用的，G 指令集并不是独立扩展，而是基础整数指令集（I）与常用扩展模块的标准组合包，旨在简化高性能通用处理器的指令集定义。其核心构成包括了基础整数运算和控制流指令的 I 指令集、硬件乘除法指令的 M 指令集、原子操作的 A 指令集、单精度浮点运算的 F 指令集和双精度浮点运算的 D 指令集，它们一起构成了 G 指令集。而 RISC-V 指令集的 G 指令集，和上文提到的 RV64 和 RV32 架构结合，就得到了面向 64 位架构的 RV64G 和面向 32 位架构的 RV32G。

因此，OpenJDK 要面向 RISC-V 指令集，最终将落实在 RV64G 和 RV32G 上。通过对 RV64G 和 RV32G 的支持，就可以支持绝大多数 RISC-V 指令集架构设备。OpenJDK 对于 RV64G 的支持，已经在社区进行了展开，但是对于 RV32G 的支持，还未有实现。

RISC-V 核心开发组件中 OpenJDK 对 RISC-V 架构的支持需求总

结如下表所示。

表 4 OpenJDK 的功能需求清单

序号	模块	子模块	需求	需求优先级	需求描述
1	OpenJDK	OpenJDK RV64G	OpenJDK RV64G 支持现状调研	高	对于 OpenJDK 目前的开发版本中，对于 RV64G 指令集的支持情况做详细的调研。
			RV64G 开发板支持	高	调研市场上目前已经量产的可以支持 RV64G 的开发板，然后使用 OpenJDK 测试其支持。
		OpenJDK RV32G	OpenJDK 32 位版本调研	高	调研目前市场上使用比较多的 OpenJDK 32 位版本，为后续选择移植基线做好准备。
			OpenJDK 32 位版本基线选	高	比较目前的 OpenJDK 32 位版

			择		本，从中选择一个版本作为移植的基线版本，为后续移植工作做好准备。
			OpenJDK RV32G 移植 内容调研	高	调 研 OpenJDK RV32G 移植所需要修改的内容范围，包含新增的文件以及原有代码的修改，确定整个移植所涉及到的范围。
			OpenJDK RV32G Zero 解释器移植	高	OpenJDK 的 Zero 解释器虽然具有很强的跨平台能力，不需要专门面向指令集的代码，但是针对 RISC-V RV32G，仍然需要对其进行适当的代码调整和测试。

2.5 QEMU 功能需求综述

QEMU (Quick Emulator) 是一款开源的硬件虚拟化工具，支持全系统模拟 (Full System Emulation) 和用户模式模拟 (User-mode Emulation)。它通过动态二进制翻译 (Dynamic Binary Translation, DBT) 技术，能够跨平台模拟多种 CPU 架构 (如 x86、ARM、RISC-V、MIPS 等)，允许用户在一种硬件架构上运行另一种架构的操作系统和程序。

QEMU 的主要功能包括：

- (1) **全系统模拟**：模拟完整的计算机系统，包括 CPU、内存、外设 (如磁盘、网络、GPU 等)，可运行未经修改的操作系统 (如 Linux、Windows)。
- (2) **用户模式模拟**：直接在宿主机上运行不同架构的应用程序，适用于跨平台开发和测试。
- (3) **硬件加速**：与 KVM (Kernel-based Virtual Machine) 结合，提供接近原生的虚拟化性能。
- (4) **设备仿真**：支持多种虚拟设备 (如网卡、USB 控制器)，并可自定义硬件模型。
- (5) **调试支持**：提供 GDB 集成、内存快照和逆向调试功能，适用于固件和操作系统开发。

QEMU 广泛应用于嵌入式开发、云计算 (如 QEMU-KVM 组合)、跨平台测试及安全研究，是 RISC-V 等新兴架构生态的关键工具。

QEMU 的全系统模拟（Full System Emulation）功能支持广泛的 CPU 架构，涵盖主流处理器、嵌入式芯片及新兴指令集。例如 x86/x86_64、ARM、PowerPC、RISC-V、MIPS、SPARC 等。全系统模拟的典型应用场景有跨平台开发，例如在 x86 主机上运行 ARM/RISC-V 的 Ubuntu 进行测试；固件调试，例如模拟 MIPS 路由器环境分析漏洞；历史系统保留，例如运行 PowerPC 版的 Mac OS 9；教育研究，例如通过 RISC-V 模拟器学习计算机体系结构等。

QEMU 的全系统模拟功能中 RISC-V 架构的支持仍处于发展阶段，部分扩展指令集的支持仍不完善或无支持，本课题拟对 B、K、P、Zce、Zfinx 扩展指令集进行支持，实现 QEMU 在全系统模拟中对 RISC-V 架构的 B、K、P、Zce、Zfinx 扩展指令集的支持。

QEMU 的用户模式模拟（User-mode Emulation）功能允许直接在宿主系统上运行不同架构的用户态程序（无需模拟完整操作系统），支持的 CPU 架构有 x86、ARM、Aarch64、PowerPC、RISC-V 等等。运行时需历来目标架构的库文件。用户模式模拟不支持内核模块加载、设备驱动、特权指令（如 MMU 操作），性能上因为采用动态二进制翻译（TCG）技术，会导致 20%-50%性能下降，对比原生运行。用户模式模拟是 QEMU 轻量级跨架构运行的核心功能，适用于无需完整 OS 环境的开发测试场景。

QEMU 的用户模式模拟功能中 RISC-V 架构的支持仍处于发展阶段，主要需要对非特权扩展指令集进行支持，本课题拟对 B、K、P、Zce、Zfinx 扩展指令集进行支持，实现 QEMU 在用户模式模拟中对

RISC-V 架构的 B、K、P、Zce、Zfinx 扩展指令集的支持。

RISC-V 核心开发组件中 QEMU 对 RISC-V 架构的本年度支持需求总结如下表所示。

表 5 QEMU 的功能需求清单

序号	模块	子 模 块	需求	需求优 先级	需求描述
1	QEMU	QEMU 全 系 统 模 拟	QEMU 在全系统模拟中对 RISC-V 架构的 B 扩展指令集的支持。	高	QEMU 在全系统模拟（Full System Emulation）中对 RISC-V 架构的 B（位操作）扩展指令集的支持
		QEMU 用 户 模 式 模拟	QEMU 在用户模式模拟中对 RISC-V 架构的 B 扩展指令集的支持。	高	QEMU 在用户模式模拟（User-mode Emulation）中对 RISC-V 架构的 B（位操作）扩展指令集的支持

2.6 OpenCV 功能需求综述

OpenCV（Open Source Computer Vision Library）作为计算机视

觉领域的核心开源库，自 1999 年由 Intel 发起以来，已成为工业界和学术界的事实标准。它提供了超过 2500 个优化算法，涵盖图像处理、特征检测、目标识别、3D 重建等全领域视觉任务，广泛应用于自动驾驶、工业检测、医疗影像等关键场景。随着 RISC-V 架构在边缘计算、嵌入式系统和服务器领域的崛起，OpenCV 亟需深度适配 RISC-V 向量扩展（RVV）架构，以充分发挥其可变长向量计算的优势。

随着 RISC-V 架构在边缘计算、嵌入式视觉和服务器级应用中的快速普及，OpenCV 亟需深度适配 RISC-V 向量扩展（RVV）架构，以满足日益增长的异构计算需求。当前 OpenCV 的 Universal Intrinsic 框架虽然已获得 RISC-V 相关支持，但是主要面向类似 x86/ARM 等固定长度向量架构设计，无法充分发挥 RISC-V 可变长向量架构的独特优势，这已成为制约 RISC-V 视觉应用性能的关键瓶颈。

当前 OpenCV 在 RISC-V 平台面临三重核心挑战：首先，RISC-V 的可变长向量架构与传统固定长度 SIMD 架构存在本质差异，现有优化方案无法充分发挥 RVV 的性能优势；其次，RISC-V 生态存在一些碎片化问题，规范版本仍在迭代中，且部分编译器支持不完善，不同工具链对 RVV 规范的支持存在差异；最后，现有实现存在性能优化空间，需要针对 RVV 架构特性进行深度适配。

在 RISC-V 生态中，OpenCV 承担着承上启下的关键作用：向下需要适配多样化的从嵌入式到服务器级的硬件平台，向上需为视觉算法提供统一高效的编程接口。当前面临的核心技术挑战包括：

(1) 架构适配挑战：RVV 的可变长向量模型与传统 SIMD 架构

存在本质差异，现有基于固定长度向量的实现方案无法充分发挥 RVV 的性能优势。这要求重构 Universal Intrinsic 框架，建立动态向量长度感知机制，实现真正的可变长向量操作。

(2) **生态碎片化问题：**RVV 规范从 0.7.1 版本经过多版更新迭代至 1.0 版本，不同硬件平台支持程度各异。需要维护 Universal Intrinsic 框架对 RVV 规范的官方支持。重点确保框架与最新 RVV 规范保持同步，及时适配规范更新；建立标准化的接口规范，为开发者提供一致的编程体验；通过编译时机制实现不同硬件平台的兼容支持。同时，GCC 与 Clang 两大编译器对 RVV 的支持存在差异性和不完善之处，导致跨平台开发面临兼容性问题。建立主流编译器的深度适配方案。针对 GCC 工具链优化指令调度逻辑；完善 Clang 编译器的关键指令支持；设计 ABI 保护层防止系统调用导致的寄存器破坏风险。

(3) **性能优化瓶颈：**现有实现中存在大量冗余内存访问和类型转换操作，无法有效利用 RVV 特有的 LMUL 寄存器分组机制，导致在关键视觉算子上难以达到硬件理论性能。

基于上述挑战，本项目提出四大核心功能需求方向：

(1) **可变长向量架构重构：**核心任务是重建 Universal Intrinsic 框架对可变长向量的原生支持。包括实现可变长向量操作技术，消除不必要的数据复制开销；构建动态元数据管理系统，支持运行时准确获取硬件向量特性；设计自适应策略，根据硬件能力自动选择最优执行方案。通过引入类型别名机制替代传统包装类，彻底消除

冗余内存访问；建立动态元数据管理能力，实现运行时向量元素数量的准确获取；深度优化 LMUL 寄存器分组策略，支持 1-8 组寄存器合并操作。该重构将使卷积等核心算子在 RISC-V 平台获得 3 倍以上的性能提升，满足工业视觉系统对实时性的严苛要求。

(2) **规范迭代的持续维护：**RISC-V 向量扩展（RVV）规范的持续演进是 OpenCV 维护工作面临的核心挑战。随着 RVV 规范从 0.7.1 向 1.0 及更高版本迭代，指令集语义、寄存器管理机制和内存模型等关键要素均发生实质性变更。这种规范演进要求 OpenCV 的 Universal Intrinsic 框架必须建立动态响应机制，以同步适配新规范特性并保持历史兼容性。

(3) **全工具链兼容保障：**建立 GCC 与 Clang 双工具链深度适配方案，实现并维护对主流开源工具链的广泛兼容性，确保 OpenCV 能在多样化的 RISC-V 开发环境中成功构建和运行。

(4) **充分发挥硬件性能：**确保 RVV 后端实现能够最大化利用底层硬件的计算能力。后端设计必须彻底消除不必要的内存操作和数据类型转换开销，并深度适配 RVV 架构特有的高性能特性，特别是其可扩展的寄存器分组机制（如 LMUL）。最终目标是在图像处理、特征提取等关键视觉计算核心上，实现接近硬件理论峰值的高效运算性能。

RISC-V 核心开发组件中 OpenCV 对 RISC-V 架构的支持面临的核心技术挑战集中于弥合架构差异与应对生态复杂性两大维度。一方面，RISC-V RVV 独特的可变长向量计算模型与传统固定长度

SIMD 架构存在根本性差异，要求对 OpenCV 的核心向量抽象层（Universal Intrinsics）进行深度重构。另一方面，RVV 规范仍处于快速迭代期，且不同硬件平台和编译器（特别是 GCC 与 Clang）对规范的支持存在显著碎片化与不完善，导致跨平台兼容性与性能优化面临严峻挑战。这些因素共同构成了在 RISC-V 平台上实现高性能、稳定可靠计算机视觉处理的核心障碍。

针对上述挑战，项目需实现四大核心功能目标：架构适配重构，即彻底改造向量抽象层以原生支持 RVV 可变长向量操作，消除数据转换与复制瓶颈；规范同步机制，建立灵活框架以持续适配 RVV 规范演进，保障长期兼容性；工具链兼容保障，深度优化并维护对 GCC 与 Clang 两大主流编译器的全面支持，确保稳定构建与运行；硬件性能释放，深度集成 RVV 高级特性（如 LMUL 寄存器分组），最大化消除冗余操作，使关键视觉算子在 RISC-V 平台上达到逼近硬件理论峰值的计算效率。

这些优化将使 OpenCV 成为 RISC-V 视觉处理的事实标准，推动 RISC-V 在工业自动化、智能安防、自动驾驶等关键领域的规模化应用，加速 RISC-V 生态从硬件创新向应用落地的价值转化。

RISC-V 核心开发组件中 OpenCV 对 RISC-V 架构的支持需求总结如下表所示。

表 6 OpenCV 的功能需求清单

序号	模块	子模块	需求	需求优先级	需求描述
----	----	-----	----	-------	------

1	OpenCV	可变长向量架构适配	可变长向量架构重构	高	重构 Universal Intrinsic 框架以原生支持 RVV 可变长向量模型，消除数据转换与复制瓶颈
			动态元数据管理系统实现	高	构建运行时硬件向量特性感知机制，支持动态获取向量长度等关键参数
			LMUL 寄存器分组优化	中	深度优化 LMUL 寄存器分组策略 (1-8 组)，最大化寄存器合并操作效率
		RVV 规范版本兼容性	RVV 规范同步更新	高	持续适配 RVV 规范演进，保障长期兼容性
		GCC/clang 工具链适配	GCC 工具链适配	高	针对 GCC 工具链优化 OpenCV 适配
			Clang 工具链适配	高	针对 clang 工具链优化 OpenCV 适配

		OpenCV 性能优化	冗余内存访问消除	高	彻底消除不必要的内存操作和数据类型转换开销
--	--	----------------	----------	---	-----------------------

三、详细功能需求

3.1 GCC 详细功能需求

3.1.1 功能需求 1：在 Binutils 中添加对 RISC-V B 扩展的汇编支持

- 1) 需求 1 优先级：P1
- 2) 功能 1 需求描述

功能编号	3.1.1
功能名称	在 Binutils 中添加对 RISC-V B 扩展的汇编支持
使用场景	嵌入式开发：在 RISC-V MCU/SoC 中使用 B 扩展指令优化位操作（如加密、压缩算法）。 高性能计算：利用 B 扩展加速位运算密集型任务（如哈希计算、数据编解码）。 编译器工具链开发：支持 GCC/LLVM 生成的 B 扩展指令的汇编和链接。 教学与研究：学习 RISC-V 扩展指令集或开发自定义指令扩展。
用户角色	嵌入式工程师，编译器开发者，FPGA/芯片设计者，学生/研究者
先决条件	Binutils 需支持 RISC-V 基础指令集

	(RV32I/RV64I)。
输入信息	汇编源代码：包含 B 扩展指令（如 clz, andn, rori）的 .s 文件。 命令行参数：riscv64-unknown-elf-as -march=rv64ib -o output.o input.s
输入数据检测	指令合法性检查：操作数是否匹配，立即数范围是否有效。 架构兼容性检查：若目标平台不支持 B 扩展，报错提示。 伪指令展开：验证高级伪指令（如 not）是否能正确转换为 B 扩展指令。
异常处理	语法错误、兼容性错误、硬件不支持
输出信息	目标文件（.o）、调试信息、日志信息

3.1.2 功能需求 2：在 Binutils 中添加对 RISC-V K 扩展的汇编支持

- 1) 需求 2 优先级：P1
- 2) 功能 2 需求描述

功能编号	3.1.2
功能名称	在 Binutils 中添加对 RISC-V K 扩展的汇编支持
使用场景	密码学加速：在安全协议（TLS/SSL、AES、SHA）中利用 K 扩展指令（如 aes32esi、sha256sig0）进行硬件加速。 嵌入式安全设备：开发安全芯片、TPM（可信平台模

	块)或智能卡固件。 操作系统内核：在内核加密模块（如 Linux Crypto API）中直接调用 K 扩展指令。 编译器工具链开发：支持 GCC/LLVM 生成的 K 扩展指令汇编与链接。 密码学研究：快速验证加密算法在硬件上的实现效果。
用户角色	安全工程师，嵌入式工程师，编译器开发者，硬件验证工程师，学生/研究者
先决条件	Binutils 需支持 RISC-V 基础指令集（RV32I/RV64I）。
输入信息	汇编源代码：包含 K 扩展指令的.s 文件。 命令行参数：<code>riscv64-unknown-elf-as -march=rv64imafdc_k -o crypto.o crypto.s</code>
输入数据检测	指令合法性检查：操作数是否匹配，立即数范围是否有效。 架构兼容性检查：若目标平台不支持 K 扩展，报错提示。 安全模式检查（可选）：某些指令（如 sm4ed）可能需要特权级（Machine Mode）支持。
异常处理	语法错误、兼容性错误、硬件不支持、特权级错误（可选）
输出信息	目标文件（.o）、调试信息、日志信息

3.1.3 功能需求 3：在 Binutils 中添加对 RISC-V P 扩展的汇编支持

- 1) 需求 3 优先级：P1
- 2) 功能 3 需求描述

功能编号	3.1.3
功能名称	在 Binutils 中添加对 RISC-V P 扩展的汇编支持
使用场景	数字信号处理（DSP）：加速音频/图像处理（如 FFT、FIR 滤波） 嵌入式 AI 推理：优化低精度神经网络计算（如 8-bit 矩阵乘） 多媒体编解码：视频压缩（H.264/HEVC）中的 SIMD 并行计算 科学计算：向量化数值运算（如雷达信号处理） 编译器开发：支持自动向量化生成的 P 扩展指令
用户角色	DSP 工程师，AI 加速器开发者，嵌入式工程师，编译器开发者，FPGA 验证工程师，学生/研究者
先决条件	Binutils 需支持 RISC-V 基础指令集（RV32I/RV64I）。
输入信息	汇编源代码：包含 P 扩展指令的.s 文件。 命令行参数： <code>riscv64-unknown-elf-as -march=rv64imafdc_p -o pext.o pext.s</code>
输入数据检测	指令合法性检查：操作数是否匹配，立即数范围是否有效。 架构兼容性检查：若目标平台不支持 P 扩展，报错提

	示。
异常处理	语法错误、兼容性错误、硬件不支持
输出信息	目标文件（.o）、调试信息、日志信息

3.1.4 功能需求 4：在 Binutils 中添加对 RISC-V Zce 扩展的汇编支持

- 1) 需求 4 优先级：P1
- 2) 功能 4 需求描述

功能编号	3.1.4
功能名称	在 Binutils 中添加对 RISC-V Zce 扩展的汇编支持
使用场景	嵌入式系统开发：为资源受限设备（IoT/边缘计算）减少代码体积（典型节省 15-30%） 实时操作系统（RTOS）：优化中断处理例程和上下文切换的指令密度 Bootloader 开发：在有限 ROM 空间内实现更复杂启动逻辑 编译器优化：支持 GCC/LLVM 生成的 Zce 压缩指令序列 教学演示：展示 RISC-V 代码密度优化技术
用户角色	RTOS 工程师，嵌入式工程师，编译器开发者，芯片验证工程师，学生/研究者
先决条件	Binutils 需支持 RVC（压缩指令）基础（Zca 子集）
输入信息	汇编源代码：包含 Zce 扩展指令的.s 文件。 命令行参数： riscv64-unknown-elf-as -

	march=rv64imafdc_zce -o zcext.o zcext.s
输入数据检测	指令合法性检查：操作数是否匹配，立即数范围是否有效。 架构兼容性检查：若目标平台不支持 Zce 扩展，报错提示。 子集依赖：编译器是否开启必要子集 对齐要求：跳转目标是否 2-byte 对齐
异常处理	语法错误、兼容性错误、硬件不支持
输出信息	目标文件（.o）、调试信息、日志信息

3.1.5 功能需求 5：在 Binutils 中添加对 RISC-V Zfinx 扩展的汇编支持

- 1) 需求 5 优先级：P1
- 2) 功能 5 需求描述

功能编号	3.1.5
功能名称	在 Binutils 中添加对 RISC-V Zfinx 扩展的汇编支持
使用场景	嵌入式 FPU 设计：在无独立浮点寄存器（F 扩展）的 RISC-V 核中实现浮点运算 实时控制系统：空间受限场景（如 MCU）下的浮点计算加速 混合精度计算：与 Zfh（半精度）扩展协同工作 教学研究：展示共享寄存器架构的浮点实现方案 编译器开发：支持 GCC/LLVM 生成的 Zfinx 指令序列
用户角色	芯片设计者，嵌入式工程师，编译器开发者，FPGA

	开发者，学生/研究者
先决条件	Binutils 需已支持 RISC-V 基础整数指令集（RV32I/RV64I）
输入信息	汇编源代码：包含 Zfinx 扩展指令的.s 文件。 命令行参数： <code>riscv64-unknown-elf-as -march=rv64imafdc_zfinx -o zfinx.o zfinx.s</code>
输入数据检测	指令合法性检查：操作数是否匹配，立即数范围是否有效。 寄存器检测：禁止使用 f0-f31 对齐要求：单精度需 32-bit 对齐 CSR 依赖：浮点状态寄存器访问
异常处理	语法错误、兼容性错误、硬件不支持
输出信息	目标文件（.o）、调试信息、日志信息

3.2 LLVM 详细功能需求

3.2.1 功能需求 1：在汇编模块中添加对 RISC-V B 扩展的汇编支持

- 1) 需求 1 优先级：P1
- 2) 功能 1 需求描述

功能编号	3.2.1
功能名称	在汇编模块中添加对 RISC-V B 扩展的汇编支持
使用场景	嵌入式开发：在 RISC-V MCU/SoC 中使用 B 扩展指令优化位操作（如加密、压缩算法）。 高性能计算：利用 B 扩展加速位运算密集型任务（如

	哈希计算、数据编解码)。 编译器工具链开发：支持 LLVM 生成的 B 扩展指令的汇编和链接。 教学与研究：学习 RISC-V 扩展指令集或开发自定义指令扩展。
用户角色	嵌入式工程师，编译器开发者，FPGA/芯片设计者，学生/研究者
先决条件	汇编模块需支持 RISC-V 基础指令集 (RV32I/RV64I)。
输入信息	汇编源代码：包含 B 扩展指令（如 clz, andn, rori）的 .s 文件。 命令行参数：llvm-mc -arch=riscv64 -mattr=+b -filetype=obj -o output.o input.s
输入数据检测	指令合法性检查：操作数是否匹配，立即数范围是否有效。 架构兼容性检查：若目标平台不支持 B 扩展，报错提示。 伪指令展开：验证高级伪指令（如 not）是否能正确转换为 B 扩展指令。
异常处理	语法错误、兼容性错误、硬件不支持
输出信息	目标文件 (.o)、调试信息、日志信息

3.2.2 功能需求 2：在汇编模块中添加对 RISC-V K 扩展的汇编支持

1) 需求 2 优先级：P1

2) 功能 2 需求描述

功能编号	3.2.2
功能名称	在汇编模块中添加对 RISC-V K 扩展的汇编支持
使用场景	密码学加速：在安全协议（TLS/SSL、AES、SHA）中利用 K 扩展指令（如 aes32esi、sha256sig0）进行硬件加速。 嵌入式安全设备：开发安全芯片、TPM（可信平台模块）或智能卡固件。 操作系统内核：在内核加密模块（如 Linux Crypto API）中直接调用 K 扩展指令。 编译器工具链开发：支持 LLVM 生成的 K 扩展指令汇编与链接。 密码学研究：快速验证加密算法在硬件上的实现效果。
用户角色	安全工程师，嵌入式工程师，编译器开发者，硬件验证工程师，学生/研究者
先决条件	LLVM 汇编模块需支持 RISC-V 基础指令集（RV32I/RV64I）。
输入信息	汇编源代码：包含 K 扩展指令的.s 文件。 命令行参数：llvm-mc -arch=riscv64 -mattr=+k -filetype=obj -o output.o input.s
输入数据检测	指令合法性检查：操作数是否匹配，立即数范围是否有效。 架构兼容性检查：若目标平台不支持 K 扩展，报错提示。

	安全模式检查（可选）：某些指令（如 sm4ed）可能需要特权级（Machine Mode）支持。
异常处理	语法错误、兼容性错误、硬件不支持、特权级错误（可选）
输出信息	目标文件（.o）、调试信息、日志信息

3.2.3 功能需求 3：在汇编模块中添加对 RISC-V P 扩展的汇编支持

- 1) 需求 3 优先级：P1
- 2) 功能 3 需求描述

功能编号	3.2.3
功能名称	在汇编模块中添加对 RISC-V P 扩展的汇编支持
使用场景	数字信号处理（DSP）：加速音频/图像处理（如 FFT、FIR 滤波） 嵌入式 AI 推理：优化低精度神经网络计算（如 8-bit 矩阵乘） 多媒体编解码：视频压缩（H.264/HEVC）中的 SIMD 并行计算 科学计算：向量化数值运算（如雷达信号处理） 编译器开发：支持自动向量化生成的 P 扩展指令
用户角色	DSP 工程师，AI 加速器开发者，嵌入式工程师，编译器开发者，FPGA 验证工程师，学生/研究者
先决条件	LLVM 汇编模块需支持 RISC-V 基础指令集（RV32I/RV64I）。

输入信息	汇编源代码：包含 P 扩展指令的.s 文件。 命令行参数：llvm-mc -arch=riscv64 -mattr=+p -filetype=obj -o output.o input.s
输入数据检测	指令合法性检查：操作数是否匹配，立即数范围是否有效。 架构兼容性检查：若目标平台不支持 P 扩展，报错提示。
异常处理	语法错误、兼容性错误、硬件不支持
输出信息	目标文件（.o）、调试信息、日志信息

3.2.4 功能需求 4：在汇编模块中添加对 RISC-V Zce 扩展的汇编支持

- 1) 需求 4 优先级：P1
- 2) 功能 4 需求描述

功能编号	3.2.4
功能名称	在汇编模块中添加对 RISC-V Zce 扩展的汇编支持
使用场景	嵌入式系统开发：为资源受限设备（IoT/边缘计算）减少代码体积（典型节省 15-30%） 实时操作系统（RTOS）：优化中断处理例程和上下文切换的指令密度 Bootloader 开发：在有限 ROM 空间内实现更复杂启动逻辑 编译器优化：支持 LLVM 生成的 Zce 压缩指令序列 教学演示：展示 RISC-V 代码密度优化技术

用户角色	RTOS 工程师，嵌入式工程师，编译器开发者，芯片验证工程师，学生/研究者
先决条件	LLVM 汇编模块需支持 RVC（压缩指令）基础（Zca 子集）
输入信息	汇编源代码：包含 Zce 扩展指令的.s 文件。 命令行参数： <code>llvm-mc -arch=riscv64 -mattr=+zce -filetype=obj -o zcext.o zcext.s</code>
输入数据检测	指令合法性检查：操作数是否匹配，立即数范围是否有效。 架构兼容性检查：若目标平台不支持 Zce 扩展，报错提示。 子集依赖：编译器是否开启必要子集 对齐要求：跳转目标是否 2-byte 对齐
异常处理	语法错误、兼容性错误、硬件不支持
输出信息	目标文件（.o）、调试信息、日志信息

3.2.5 功能需求 5：在汇编模块中添加对 RISC-V Zfinx 扩展的汇编支持

- 1) 需求 5 优先级：P1
- 2) 功能 5 需求描述

功能编号	3.2.5
功能名称	在汇编模块中添加对 RISC-V Zfinx 扩展的汇编支持
使用场景	嵌入式 FPU 设计：在无独立浮点寄存器（F 扩展）的 RISC-V 核中实现浮点运算

	实时控制系统：空间受限场景（如 MCU）下的浮点计算加速 混合精度计算：与 Zfh（半精度）扩展协同工作 教学研究：展示共享寄存器架构的浮点实现方案 编译器开发：支持 LLVM 生成的 Zfinx 指令序列
用户角色	芯片设计者，嵌入式工程师，编译器开发者，FPGA 开发者，学生/研究者
先决条件	LLVM 汇编模块需已支持 RISC-V 基础整数指令集（RV32I/RV64I）
输入信息	汇编源代码：包含 Zfinx 扩展指令的.s 文件。 命令行参数：<code>llvm-mc -arch=riscv64 -mattr=+zfinx -filetype=obj -o zfinx.o zfinx.s</code>
输入数据检测	指令合法性检查：操作数是否匹配，立即数范围是否有效。 寄存器检测：禁止使用 f0-f31 对齐要求：单精度需 32-bit 对齐 CSR 依赖：浮点状态寄存器访问
异常处理	语法错误、兼容性错误、硬件不支持
输出信息	目标文件（.o）、调试信息、日志信息

3.3 V8 详细功能需求

3.3.1 构建系统适配

1) 需求 1 优先级

P1

2) 功能 1 需求描述

V8 是由 Google 开发的高性能 JavaScript 和 WebAssembly 引擎，广泛应用于 Chrome 浏览器、Node.js 等场景。其构建系统负责管理源代码编译流程，包括依赖管理、编译参数配置、目标平台适配等。

“对 V8 构建系统适配”指针对特定硬件架构或环境需求，对构建系统的编译逻辑、配置文件、工具链集成等进行定制化修改，使其能够生成适配目标平台的可执行代码或库文件。

3.3.2 RISC-V64 位和 32 位基础指令集的功能支持

1) 需求 2 优先级

P2

2) 功能 2 需求描述

对所要实现的需求及功能点进行描述。包括：需求使用的场景、用户角色；本功能实现的先决条件、输入信息；对输入数据的有效性检查、处理（包括异常情况处理）；输出信息及其流向、相应的提示信息。推荐用文字加图表的形式展现。

RISC-V 是一种开源的指令集架构（ISA），采用模块化设计，由基础指令集和可选扩展组成。RISCVIMFDA（建成 RISCVG 指令集）位桌面和嵌入式计算提供了最小功能集。为了在 V8 上支持 RISC-V 架构，需通过支持 RISC-V64 位（RV64IMFD/A）和 32 位（RV32IMFD/A）基础指令集，确保 JavaScript 和 WebAssembly 代码

在这些新硬件上的高效运行，覆盖从高性能服务器到低功耗 IoT 设备的全场景。

3.3.3 32/64 位运行时环境适配

RISC-V 是一种开源的指令集架构（ISA），采用模块化设计，由基础指令集和可选扩展组成。RISCVIMFDA（建成 RISCVG 指令集）位桌面和嵌入式计算提供了最小功能集。为了在 V8 上支持 RISC-V 架构，需通过支持 RISC-V 64 位（RV64IMFD/A）和 32 位（RV32IMFD/A）基础指令集，确保 JavaScript 和 WebAssembly 代码在这些新硬件上的高效运行，覆盖从高性能服务器到低功耗 IoT 设备的全场景。

3.4 OpenJDK 详细功能需求

3.4.1 功能需求 1：OpenJDK RV64G 支持现状调研。

1) 需求 1 优先级

P1

2) 功能 1 需求描述

功能编号	3.4.1
功能名称	OpenJDK RV64G 支持现状调研
使用场景	1. 编译器开发者、解释器开发者需要了解目前 OpenJDK RV64G 目前已经支持的情况，基于目前支持的情况，可以为 OpenJDK 添加新的支持，或者是验

	证目前已经支持的指令； 2. Java 用户可以根据目前 OpenJDK RV64G 支持的情况来决定是否在 RV64G 平台上使用 OpenJDK。
用户角色	编译器开发者、解释器开发者、Java 用户
先决条件	OpenJDK 社区代码库、代码库主干分支、代码库近几年的发布版本分支。
输入信息	计划在 RV64G 平台上执行的 Java 程序，OpenJDK 源码。
输入数据检测	可以正常执行的 Java 程序；OpenJDK 源码主干分支，需要支持 RV64 的基本功能。
异常处理	Java 程序不支持 RV64G 的平台正确执行；OpenJDK 源码中检索不到完整的 RV64 指令支持实现；则将该版本标记为不支持 RV64G。
输出信息	Java 程序可以在支持 RV64G 的平台正确执行；OpenJDK 源码中可以检索到完整的 RV64 指令支持实现。

3.4.2 功能需求 2：RV64G 开发板支持。

1) 需求 2 优先级

P1

2) 功能 2 需求描述

功能编号	3.4.2
功能名称	RV64G 开发板支持
使用场景	1. 编译器开发者、解释器开发者可以在 RV64G 开发板上运行 OpenJDK，验证 OpenJDK 在实际开发板

	上运行的性能和正确性； 2. Java 用户可以在 RV64G 开发板上运行 Java 程序，可以将更多的 Java 应用运行到 RV64G 开发板上。
用户角色	编译器开发者、解释器开发者、Java 用户
先决条件	已经可以支持 RV64G 的 RISC-V 开发板；可以支持 RV64G 的 OpenJDK 版本。
输入信息	可以正常运行的 Java 程序，可以支持 RV64G 的 OpenJDK 版本。
输入数据检测	Java 程序是否可以正常运行；OpenJDK 版本是否可以正常支持 RV64G。
异常处理	Java 程序在 RV64G 开发板上运行之后，未得到程序运行的正常结果，可以根据结果判定是属于程序、OpenJDK 或者是开发板的问题。
输出信息	Java 程序在 RV64G 开发板上运行之后，可以得到程序运行的正常结果。

3.4.3 功能需求 3：OpenJDK 32 位版本调研。

1) 需求 3 优先级

P1

2) 功能 3 需求描述

功能编号	3.4.3
功能名称	OpenJDK 32 位版本调研
使用场景	1. 编译器开发者、解释器开发者需要了解目前 OpenJDK 32 位版本目前支持的情况，主要发布的版本

	以及后续发展的目标，考虑是否为 OpenJDK 32 位添加新的指令集支持； 2. Java 用户可以根据目前 OpenJDK 32 位版本支持的情况来决定是否使用 OpenJDK 32 位版本。
用户角色	编译器开发者、解释器开发者、Java 用户
先决条件	多个发布版本的 OpenJDK
输入信息	多个发布版本的 OpenJDK
输入数据检测	检测多个发布版本的 OpenJDK 是否对 32 位支持，以及现有用户使用较多的版本是哪些版本。
异常处理	如果该 OpenJDK 版本不支持 32 位，则标记为不支持。
输出信息	输出可以支持 32 位的 OpenJDK 版本，并在其中列出当下用户使用较多的 OpenJDK 版本。

3.4.4 功能需求 4：OpenJDK 32 位版本基线选择。

1) 需求 4 优先级

P1

2) 功能 4 需求描述

功能编号	3.4.4
功能名称	OpenJDK 32 位版本基线选择
使用场景	1. 编译器开发者、解释器开发者根据目前 OpenJDK 32 位版本目前支持的情况，主以及用户使用 OpenJDK 32 位较多的版本，考虑将其作为 OpenJDK RV32G 的移植基线；

	2. Java 用户可以根据选定的移植基线，作为后续 OpenJDK RV32G 使用的提前选定版本，作为后续开发的选择版本。
用户角色	编译器开发者、解释器开发者、Java 用户
先决条件	选定好的多个可以支持 32 位的 OpenJDK 版本
输入信息	多个可以支持 32 位的 OpenJDK 版本
输入数据检测	检测多个可以支持 32 位的 OpenJDK 版本，确保其用户数量较多，并可以开展 RV32G 相关移植。
异常处理	如果该 OpenJDK 版本用户数量较少，或者无法移植 RV32G 版本，则排除该版本。
输出信息	输出一个最终选定的 OpenJDK 版本，作为后续移植 RV32G 的。

3.4.5 功能需求 5：OpenJDK RV32G 移植内容调研。

1) 需求 5 优先级

P1

2) 功能 5 需求描述

功能编号	3.4.5
功能名称	OpenJDK RV32G 移植内容调研
使用场景	1. 编译器开发者、解释器开发者根据目前 OpenJDK RV32G 的移植基线可以知道移植所需要的工作内容，为后续的移植工作做好准备； 2. Java 用户可以根据移植内容，知道 OpenJDK 哪部分源码是和指令集架构相关的。

用户角色	编译器开发者、解释器开发者、Java 用户
先决条件	选定好的 OpenJDK RV32G 移植基线
输入信息	OpenJDK RV32G 移植基线、RV32G 指令集、OpenJDK 源码、OpenJDK 相关文档和论文
输入数据检测	OpenJDK RV32G 的移植基线应该跟之前的工作结论相吻合，OpenJDK 源码应该符合移植基线。
异常处理	如果无法确定该部分源码是否属于移植范围，则放入移植范围内，等待移植的时候做专门的处理。
输出信息	确定针对 RV32G 进行移植所需要增加和修改的文件，对于一些修改的文件，要确定其中需要修改的具体函数。

3.4.6 功能需求 6：OpenJDK RV32G Zero 解释器移植。

1) 需求 6 优先级

P1

2) 功能 6 需求描述

功能编号	3.4.6
功能名称	OpenJDK RV32G Zero 解释器移植
使用场景	1. 编译器开发者、解释器开发者使用 Zero 解释器来验证 RV32G 相关的指令集的支持程度； 2. Java 用户可以直接使用 Zero 解释器，在 RV32G 相关平台上运行 Java 程序。
用户角色	编译器开发者、解释器开发者、Java 用户
先决条件	选定好的 OpenJDK RV32G 移植基线

输入信息	Java 程序
输入数据检测	Java 程序是否符合规范，是否可以使用 javac 进行编译。
异常处理	如果该程序无法进行正常的编译，则报错，根据报错信息重新调整输入程序。
输出信息	可以在 RV32G 平台上，正确的运行 Java 程序，并且得到正确的结果。

3.5 QEMU 详细功能需求

3.5.1 功能需求 1：QEMU 在全系统模拟中对 RISC-V 架构的 B 扩展指令集的支持。

1) 需求 1 优先级

P1

2) 功能 1 需求描述

功能编号	3.5.1
功能名称	QEMU 在全系统模拟中对 RISC-V 架构的 B 扩展指令
使用场景	1. 嵌入式系统开发：开发者在模拟环境中测试和调试使用 B 扩展指令的 RISC-V 程序； 2. 操作系统移植：在 QEMU 中运行支持 B 扩展指令的 RISC-V 操作系统； 3. 编译器验证：验证编译器生成的 B 扩展指令代码的正确性； 4. 教学与研究：计算机体系结构教学中演示 B 扩展指令的执行过程。

用户角色	嵌入式开发者、系统开发者、教育工作者、验证工程师。
先决条件	已安装支持 RISC-V 架构的 QEMU 版本，QEMU 已配置基本的 RISC-V ISA 支持（RV32I/RV64I）
输入信息	QEMU 启动参数 -cpu 或 -machine 客户机的固件、磁盘镜像、操作系统安装介质等
输入数据检测	指令格式检查：验证指令编码是否符合 B 扩展规范，检查指令是否在当前的 XLEN 下有效 特权级检查：确认当前特权级允许执行该指令，验证 CSR 访问权限 资源检查：目标寄存器是否为 x0，立即数范围是否有效
异常处理	非法指令：跳转到 mtvec/stvec，设置 mcause/scause 环境调用：特定 B 扩展指令用于系统调用，跳转到监管/机器模式 断点异常：调试模式下的 B 扩展指令，进入调试模式
输出信息	寄存器更新：目标寄存器写入指令运算结果，PC 更新为下一条指令地址 标志位更新：某些指令可能设置条件标志

3.5.2 功能需求 2：QEMU 在用户模式模拟中对 RISC-V 架构的 B 扩展指令集的支持。

1) 需求 2 优先级

P1

2) 功能 2 需求描述

功能编号	3.5.2
功能名称	QEMU 在用户模式模拟中对 RISC-V 架构的 B 扩展指令
使用场景	1. 嵌入式系统开发：开发者在模拟环境中测试和调试使用 B 扩展指令的 RISC-V 程序； 2. 操作系统移植：在 QEMU 中运行支持 B 扩展指令的 RISC-V 操作系统； 3. 编译器验证：验证编译器生成的 B 扩展指令代码的正确性； 4. 教学与研究：计算机体系结构教学中演示 B 扩展指令的执行过程。
用户角色	嵌入式开发者、系统开发者、教育工作者、验证工程师。
先决条件	已安装支持 RISC-V 架构的 QEMU 版本，QEMU 已配置基本的 RISC-V ISA 支持（RV32I/RV64I）
输入信息	QEMU 启动参数-cpu 或-machine 客户机的二进制代码：包含合法 B 扩展指令的 RISC-V 可执行文件
输入数据检测	指令格式检查：验证指令编码是否符合 B 扩展规范，检查指令是否在当前的 XLEN 下有效 特权级检查：确认当前特权级允许执行该指令，验证 CSR 访问权限 资源检查：目标寄存器是否为 x0，立即数范围是否有效

异常处理	非法指令：跳转到 mtvec/stvec，设置 mcause/scause 环境调用：特定 B 扩展指令用于系统调用，跳转到监管/机器模式 断点异常：调试模式下的 B 扩展指令，进入调试模式
输出信息	寄存器更新：目标寄存器写入指令运算结果，PC 更新为下一条指令地址 标志位更新：某些指令可能设置条件标志

3.6 OpenCV 详细功能需求

3.6.1 功能需求 1：可变长向量架构重构

1) 需求 1 优先级

P1

2) 功能 1 需求描述

功能编号	3.6.1
功能名称	可变长向量架构重构
使用场景	1. 高分辨率工业视觉实时处理（如半导体缺陷检测） 2. 自动驾驶多目标跟踪的密集计算场景 3. 医疗影像三维重建的并行体素处理
用户角色	异构计算架构师、嵌入式视觉工程师
先决条件	目标硬件支持 RVV 规范，已启用 OpenCV HAL 层优化
输入信息	任意尺寸图像/视频流 RVV 向量长度配置参数

输入数据检测	内存对齐校验 数据类型兼容性检查 寄存器组可用性检测
异常处理	非法向量长度：自动降级为标量模式并报警 寄存器溢出：触发分块处理机制 数据依赖冲突：插入显式内存屏障指令
输出信息	优化后的向量化计算结果 执行性能分析报告 向量利用率统计

3.6.2 功能需求 2：动态元数据管理系统实现

1) 需求 2 优先级

P1

2) 功能 2 需求描述

功能编号	3.6.2
功能名称	动态元数据管理系统实现
使用场景	1. 跨平台视觉应用部署 2. 混合精度计算资源动态分配 3. 自适应负载均衡流水线
用户角色	系统部署工程师、性能优化专家
先决条件	目标硬件支持 RVV 规范，已启用 OpenCV HAL 层优化
输入信息	vlenb 寄存器值 微架构特性标志

输入数据检测	内存对齐校验 数据类型兼容性检查 寄存器组可用性检测
异常处理	非法向量长度：自动降级为标量模式并报警 寄存器溢出：触发分块处理机制 数据依赖冲突：插入显式内存屏障指令
输出信息	最优向量配置参数 硬件能力分析报告

3.6.3 功能需求 3：LMUL 寄存器分组优化

1) 需求 3 优先级

P2

2) 功能 3 需求描述

功能编号	3.6.3
功能名称	LMUL 寄存器分组优化
使用场景	1. 大卷积核滤波 2. 高维特征张量变换 3. 点云数据处理中的矩阵变换
用户角色	HPC 工程师、算法优化专家
先决条件	目标硬件支持 RVV 规范，已启用 OpenCV HAL 层优化
输入信息	矩阵维度 数据块大小
输入数据	寄存器分组冲突

检测	指令流水间隙 内存对齐
异常处理	寄存器溢出：分块处理+滑动窗复用 数据未对齐：触发内存重排缓冲区 流水线阻塞：插入 vsetvli 重配置指令
输出信息	加速比报告 寄存器利用率 内存带宽占用降幅

3.6.4 功能需求 4：RVV 规范同步更新

1) 需求 4 优先级

P1

2) 功能 4 需求描述

功能编号	3.6.4
功能名称	RVV 规范同步更新
使用场景	1. 混合硬件平台 2. 规范渐进升级 3. 历史算法迁移
用户角色	框架维护工程师、芯片验证工程师
先决条件	目标硬件支持 RVV 规范，已启用 OpenCV HAL 层优化
输入信息	RVV 规范版本标识符 指令集差异映射表 硬件特性检测结果

输入数据检测	指令语义兼容性 寄存器管理机制差异 内存模型一致性
异常处理	未定义指令：捕获 ILLEGAL_INSTRUCTION 异常 版本冲突：启用指令模拟层 语义不一致：动态重写为等效指令序列
输出信息	运行时指令集调度方案 版本兼容性报告 性能损耗日志

3.6.5 功能需求 5：GCC 工具链适配

1) 需求 5 优先级

P1

2) 功能 5 需求描述

功能编号	3.6.5
功能名称	GCC 工具链适配
使用场景	1. 边缘设备部署 2. 高性能服务器优化 3. 编译器验证
用户角色	编译优化工程师、嵌入式开发工程师
先决条件	目标硬件支持 RVV 规范，已启用 OpenCV HAL 层优化 GCC 12.1+ & RISC-V 工具链 2024.04+
输入信息	向量化算法源码

	编译参数 目标硬件特性
输入数据检测	指令延迟模型分析 分支预测失效检测 内联汇编约束校验
异常处理	非法内联汇编：触发编译错误 指令调度冲突：自动插入 vsetvli 重同步指令 寄存器分配失败：降级为标量代码生成
输出信息	优化后二进制文件 代码密度报告 IPC 提升分析

3.6.6 功能需求 6：Clang 工具链适配

1) 需求 6 优先级

P1

2) 功能 6 需求描述

功能编号	3.6.6
功能名称	Clang 工具链适配
使用场景	1. 跨平台编译 2. 实时 JIT 优化 3. 调试支持
用户角色	工具链开发工程师、系统架构师
先决条件	目标硬件支持 RVV 规范，已启用 OpenCV HAL 层优化

	LLVM 15.0+
输入信息	向量寄存器分配参数 ABI 规范文件 调试符号表
输入数据检测	寄存器别名冲突检查 内存对齐约束 指令组合有效性
异常处理	V 寄存器污染：备份/恢复机制自动激活 ABI 违规：终止进程并输出 指令不支持：回退至 LLVM 标量后端
输出信息	ABI 合规性验证报告 向量寄存器使用热力图 指令吞吐量分析

3.6.7 功能需求 7：冗余内存访问消除

1) 需求 7 优先级

P1

2) 功能 7 需求描述

功能编号	3.6.7
功能名称	冗余内存访问消除
使用场景	1. 视频稳像 2. 图像金字塔 3. 特征匹配
用户角色	内存优化工程师、算法加速专家

先决条件	目标硬件支持 RVV 规范，已启用 OpenCV HAL 层优化
输入信息	图像数据流地址 数据依赖图 内存访问模式
输入数据检测	访存重叠分析 缓存命中率检测 数据类型转换开销
异常处理	缓存穿透：启用显式预取 地址未对齐：触发内存重排缓冲区 带宽超限：启用数据压缩
输出信息	内存带宽占用降幅 缓存命中率提升报告 执行周期分析

四、 非功能需求

4.1 GCC 非功能需求

4.1.1 性能需求

- 1) 编译工具链能够利用扩展指令集特性对程序进行代码体积优化，相比优化前代码体积缩减 5%以上；
- 2) 编译工具链能够利用扩展指令集特性对程序进行性能优化，相比优化前性能提升 5%以上。

4.1.2 安全需求

- 1) 指令集安全验证：确保硬件指令实现与标准完全一致，防止因指令语义偏差导致的安全漏洞；
- 2) 特权隔离保护：严格区分用户模式与特权模式指令（如 K 扩展的 `aes64ksli` 仅限 Machine Mode 使用），防止用户程序越权访问敏感硬件资源；
- 3) 抗侧信道攻击：密码学指令（如 AES/SHA）的代码生成必须保证时序恒定（Constant-Time），防止通过执行时间差泄露密钥信息。

4.1.3 兼容性需求

- 1) 向后二进制兼容性

新版本工具链生成的目标文件必须能在旧版本硬件上正确运行（需降级处理），确保芯片迭代时不影响现有固件生态。

- 2) 跨工具链协同

GCC 生成的目标文件需与 LLVM/Binutils 工具链完全兼容，避免混合工具链场景下的链接/调试失败。

- 3) 扩展组合兼容性

支持不同标准扩展组合，例如 `rv32imac_zce`，`rv64gc_k_p` 等，满足不同应用场景的定制化需求。

4.1.4 外部接口需求

(1) 用户界面（命令行接口 CLI）

RISC-V GCC 的命令行接口严格遵循 GNU 工具链规范，通过 `-march` 参数指定目标架构及扩展指令集（如 `-march=rv64imafdc_k` 启用密码学扩展），`-mtune` 参数微调目标处理器优化策略，并支持 `f[no-]` 系列标志控制扩展特性（如 `-fno-riscv-attribute` 禁用属性生成）。环境变量 `RISCV_ARCH_EXTENSIONS` 可覆盖默认扩展配置，确保与构建系统的灵活集成。该设计通过显式架构声明和结构化错误反馈，平衡了扩展指令集的灵活性与兼容性需求。

(2) 硬件接口

RISC-V GCC 的硬件接口要求目标平台完整支持 RISC-V RV32/RV64 基础指令集及指定的扩展指令（如 B/K/P/Zce/Zfinx 等），包括整数寄存器 `x0-x31`（Zfinx 扩展需复用整型寄存器进行浮点运算）、控制状态寄存器（CSR）以及 RVWMO 弱内存顺序模型。硬件需提供足够的执行单元以处理扩展指令（如密码学加速器或 SIMD 单元），并兼容标准调试接口（如 JTAG/SDT 协议）以实现扩展指令的调试支持。最低支持 RV32I 基础架构（无乘除法单元），推荐配置为 RV64GC 架构及扩展指令加速器，确保指令执行与内存访问符合 RISC-V 规范，同时需正确处理非法指令异常等硬件交互场景。

(3) 软件接口

RISC-V GCC 的软件接口要求 GCC 与关键工具链组件无缝协作：通过标准 ELF 文件与 Binutils 交互指令编码和重定位信息，确保汇编

/链接兼容性；遵循 RISC-V ABI 规范与 glibc/newlib 共享浮点/原子操作等运行时支持；生成包含扩展指令调试信息的 DWARF 格式供 GDB 解析；同时支持与 LLVM 工具链交叉验证优化结果。接口版本需匹配 glibc $\geq$ 2.33 和 QEMU $\geq$ 6.0 等依赖组件，通过属性段 (.riscv.attributes) 声明扩展要求，并在编译器、汇编器、调试器之间传递指令语义、寄存器分配等关键元数据，形成端到端的扩展指令处理链路。

4.1.5 可维护性需求

- 1) 模块化设计：扩展指令支持应封装为独立模块，便于新增/修改指令集。
- 2) 版本兼容性：扩展指令支持应封装为独立模块，便于新增/修改指令集。
- 3) 测试与验证：每个扩展需配套完整测试用例，覆盖功能、性能和边界条件。
- 4) 文档与注释：代码需包含详细注释，并维护开发者文档。

4.1.6 可移植性需求

- 1) 跨操作系统支持：工具链需在主流操作系统（Linux、Windows、macOS）上编译和运行。
- 2) 硬件架构无关性：GCC 后端逻辑应与主机架构（x86/ARM 等）解耦，确保在非 RISC-V 机器上交叉编译。

- 3) 构建系统兼容性：支持多种构建系统（Make、CMake、Meson）集成。
- 4) 运行时库可移植性：生成的代码应能在不同 RISC-V 硬件实现（如 SiFive、Andes、自定义核）上运行。

4.2 LLVM 非功能需求

4.2.1 性能需求

- 1) 编译工具链能够利用扩展指令集特性对程序进行代码体积优化，相比优化前代码体积缩减 5%以上；
- 2) 编译工具链能够利用扩展指令集特性对程序进行性能优化，相比优化前性能提升 5%以上。

4.2.2 安全需求

- 1) 指令集安全验证：确保硬件指令实现与标准完全一致，防止因指令语义偏差导致的安全漏洞；
- 2) 特权隔离保护：严格区分用户模式与特权模式指令（如 K 扩展的 aes64ks1i 仅限 Machine Mode 使用），防止用户程序越权访问敏感硬件资源；
- 3) 抗侧信道攻击：密码学指令（如 AES/SHA）的代码生成必须保证时序恒定（Constant-Time），防止通过执行时间差泄露密钥信息。

4.2.3 兼容性需求

1) 向后二进制兼容性

新版本工具链生成的目标文件必须能在旧版本硬件上正确运行（需降级处理），确保芯片迭代时不影响现有固件生态。

2) 跨工具链协同

LLVM 生成的目标文件需与 GCC/Binutils 工具链完全兼容，避免混合工具链场景下的链接/调试失败。

3) 扩展组合兼容性

支持不同标准扩展组合，例如 rv32imac_zce, rv64gc_k_p 等，满足不同应用场景的定制化需求。

4.2.4 外部接口需求

1) 用户界面（命令行接口 CLI）

用户界面接口需求主要围绕 LLVM 工具链的命令行交互和 API 设计，要求严格遵循 LLVM 现有的参数规范和错误处理标准。在命令行层面，必须支持通过 `-march` 和 `-mattr` 参数显式指定 RISC-V 扩展指令集（如 `+v,+b`），并采用统一的 `--target=riscv64-unknown-elf` 目标三元组格式；错误输出需符合 LLVM 标准格式 `<文件>:<行>:<列>:<类型>:<描述>`，确保与现有工具链兼容。对于链接阶段，需支持通过 `-L` 和 `-l` 参数传递库路径和名称，并与 LLVM 链接器 `lld` 无缝集成，生成符合 ELF 标准的输出文件。该设计强调显式声明、一致性以及向后兼容性，避免隐式行为，同时需保持与 RISC-V 官方工具链的

ABI 兼容。

2) 硬件接口

硬件接口需求要求 LLVM 后端必须兼容 RISC-V 标准扩展指令集（如 V/B/K 等）的硬件实现，能够正确识别和处理目标处理器的扩展功能特性（如向量寄存器、位操作指令等）。在代码生成阶段，需要将指令集扩展标志（如 RV64IMAFDCB）准确传递至各优化环节，并支持对扩展专用寄存器（如向量寄存器 v0-v31）的分配管理。同时，后端需遵循 RISC-V 特权架构规范（如 v1.12）处理特权指令和硬件特性，确保生成的机器码能正确在目标硬件上执行，但不需要实现额外的硬件通信协议或外部设备交互功能。该设计需保持硬件特性的透明传递，不引入硬件相关的编译时依赖。

3) 软件接口

软件接口需求要求 LLVM 后端必须与 LLVM 核心模块（如 LLVMCore、LLVMTarget）保持 API 兼容，支持通过标准 LLVM IR 传递目标架构和扩展指令集属性，并能够与 Clang 前端、llvm-mc 汇编器和 lld 链接器无缝协同工作。后端需要处理从优化后的 IR 到机器码的完整转换流程，包括指令选择、寄存器分配和代码生成，同时集成 LLVM 的 MC 框架以生成符合 ELF 格式的目标文件。该实现需兼容 LLVM 现有的测试框架（如 Lit 测试），并支持通过 compiler-rt 提供扩展指令的运行时辅助函数，确保与 RISC-V 生态系统工具链的互操作性，同时保持模块化设计以便未来扩展。

4.2.5 可维护性需求

- 1) 模块化设计：扩展指令支持应封装为独立模块，便于新增/修改指令集。
- 2) 版本兼容性：扩展指令支持应封装为独立模块，便于新增/修改指令集。
- 3) 测试与验证：每个扩展需配套完整测试用例，覆盖功能、性能和边界条件。
- 4) 文档与注释：代码需包含详细注释，并维护开发者文档。

4.2.6 可移植性需求

- 1) 跨操作系统支持：工具链需在主流操作系统（Linux、Windows、macOS）上编译和运行。
- 2) 硬件架构无关性：LLVM 后端逻辑应与主机架构（x86/ARM 等）解耦，确保在非 RISC-V 机器上交叉编译。
- 3) 构建系统兼容性：支持多种构建系统（Make、CMake、Meson）集成。
- 4) 运行时库可移植性：生成的代码应能在不同 RISC-V 硬件实现（如 SiFive、Andes、自定义核）上运行。

4.3 V8 非功能需求

4.3.1 可维护性需求

V8 的代码需遵行如下规范：

- 新增代码符合 Google 代码风格；
- 在关键算法以及重点修改的位置，添加相应的注释；
- 重要更新需要增加文档说明。

4.4 OpenJDK 非功能需求

4.4.1 可维护性需求

(1) 代码规范

- 新增代码符合 OpenJDK 代码风格；
- 修改代码符合当前文件中代码的风格，如果和代码风格有冲突，优先遵循当前文件中的代码风格；
- 在关键算法以及重点修改的位置，添加相应的注释。

(2) 文档要求

对于移植过程中遇到的重要问题，以及重要环节，形成相关的文档。

4.5 QEMU 非功能需求

4.5.1 性能需求

1) 指令执行效率

B 扩展：位操作指令（如 CLZ、ANDN）在 x86 宿主上的模拟执行速度应至少达到原生硬件性能的 70%（使用 riscv-tests 基准测试）。

K 扩展：密码学指令（如 AES64ES）在支持 AES-NI 的宿主上，加解密吞吐量应不低于 50MB/s（测试用例：OpenSSL AES-256-CBC）。

P 扩展：SIMD 指令（如 ADD8）在 AVX2 宿主上的向量化加速比应 $\geq 2x$ （对比标量实现）。

Zce 扩展：压缩指令（如 C.PUSH）应减少代码体积 15%-20%（Core-Mark 测试）。

Zfinx 扩展：单精度浮点指令在 Zfinx 模式下执行效率应达到与标准 F 扩展相比，性能下降不超过 15%。

2) 启动与响应时间

全系统模拟启动时间（到 Shell 提示符）在 i7-1185G7 宿主上不超过 10 秒（RV64GC+B 扩展，Linux 内核 5.10）。

用户模式模拟执行单个指令的平均延迟 ≤ 500 时钟周期（TCG 动态翻译模式）。

4.5.2 安全需求

(1) 数据隔离

- K 扩展的密钥相关操作（如 AES 轮密钥）需在模拟执行后清零临时寄存器。
- 禁止在用户模式模拟中执行特权指令（如 MRET），触发 SIGILL 信号。

(2) 抗侧信道攻击

密码学指令（如 SHA256SIG0）的执行时间差异不超过 10 时钟周期（防止计时攻击）。

(3) 输入验证

对非法指令编码（如未启用扩展时使用 B 扩展指令）必须触发 `illegal_instruction` 异常。

4.5.3 兼容性需求

(1) 硬件兼容性

- 支持主流宿主架构：x86_64（含 Intel/AMD）、ARM64（含 Apple M1）、RISC-V Host（需 KVM 加速）。
- 兼容 RISC-V 特权架构版本 1.12，包括 Sv39/Sv48 分页机制。

(2) 软件兼容性

- 全系统模拟需兼容以下操作系统：Linux 发行版（Fedora RISC-V、OpenSUSE Tumbleweed）
- 用户模式模拟需通过 glibc 2.33+ 和 musl 1.2.2+ 的 ABI 测试套件。

(3) 扩展协同

支持扩展组合启用（如 `rv64gc_b_k_p_zce_zfinx`），禁用冲突扩展（如 Zfinx 与 F 扩展互斥）。

4.5.4 外部接口需求

(1) 命令行接口

- 扩展启用参数格式

```
-cpu  
rv64,b=true,k=true,p=true,zce=true,zfinx=true
```

- 支持动态查询 CPU 扩展状态

(2) 调试接口

通过 GDB 10.1+支持扩展指令的单步调试，包括：反汇编显示 B/K/P 扩展指令；寄存器查看（如 Zfinx 模式下浮点值在整数寄存器中的表示）。

4.5.5 可维护性需求

(1) 代码规范

- 新增代码符合 QEMU 代码风格（Linux 内核风格，缩进 4 空格）。
- 每个扩展模块独立文件，例如 trans_rvb.c、trans_rvk.c 等。

(2) 文档要求

详细定义每个扩展支持的需求，设计说明，实现方法和测试方法。

4.5.6 可移植性需求

支持从 QEMU7.0 版本向上兼容，确保 5 年内无需重大重构。

4.6 OpenCV 非功能需求

4.6.1 性能需求

(1) 指令集优化效率

- 通过 RVV intrinsic 重写 OpenCV universal intrinsic，目标是在相同硬件条件下对比标量实现提升 50% 以上性能，重点优化循环展开与尾端处理逻辑。
- 利用动态向量长度配置实现自适应分块，减少冗余内存访问周期 30% 以上。

(2) 编译优化协同

- 启用 GCC/Clang 的 -O3 及向量化编译标志（如 -march=rv64gcv），确保优化后的代码生成效率提升 40%，避免寄存器分配冲突导致的降频。

4.6.2 安全需求

(1) 寄存器操作安全

- 运行时保护：RVV 寄存器组执行后自动清零。
- 指令语义验证：对非法指令编码触发非法指令异常。

4.6.3 兼容性需求

(1) 工具链适配

- 支持 GCC 12.1+/Clang 15.0+的 RVV 内联汇编语法，确保跨编译器性能差异不超过 15%。

(2) 硬件抽象层

- 通过 Universal Ininsics 接口封装 RVV 实现，使算子在 x86/ARM/RISC-V 平台性能波动控制在 $\pm 20\%$ 以内

4.6.4 外部接口需求

(1) 工具链接口

- 编译参数标准化:统一 RVV 使能参数，确保跨工具链一致性。

4.6.5 可维护性需求

(1) 代码架构

- 模块化设计: RVV 优化代码独立为 hal/rvv 目录，与核心模块解耦。
- 文档要求: 为 RVV 指令优化提供示例及版本迁移指南。

(2) 测试体系

- 在 QEMU RV64GCV 模式运行 OpenCV 测试集，验证行为一致性。

4.6.6 可移植性需求

(1) 跨平台支持

- 操作系统适配：同一代码库支持 Linux/Windows/Android 编译。
- 硬件抽象层：通过 Universal Ininsics 接口实现跨平台兼容性。

五、 约束要求

5.1 GCC 约束要求

5.1.1 标准符合性

1) RISC-V 规范

必须严格遵循 RISC-V ISA 官方标准（如 Volume I: Unprivileged ISA 和 Volume II: Privileged ISA）。

2) 工具链标准

遵循 GNU 工具链规范（如 ELF 格式、DWARF 调试信息），需兼容 LLVM 工具链的 RISC-V 后端（确保交叉工具链互操作）。

5.1.2 硬件约束

1) 指令执行时间：扩展指令的延迟/吞吐量需在目标硬件文档中明确标注（如 $\text{aes32esi} \leq 3$ 周期）。

2) 内存限制：编译器自身运行需 $\leq 4\text{GB}$ 内存（处理大型项目时），生成代码需支持 $\leq 64\text{KB}$ RAM 的 MCU。

- 3) 寄存器压力: Zfinx 扩展下, 浮点运算需复用整型寄存器, 避免寄存器溢出。
- 4) 对齐要求: 向量指令 (如 P 扩展) 需保证内存访问对齐 (否则触发非对齐异常)。

5.1.3 技术限制

- 1) 接口限制: 仅支持标准 RISC-V 调试接口 (JTAG/SDT), 不支持私有调试协议; 必须通过 ELF 文件传递扩展指令信息, 禁止自定义二进制格式。
- 2) 数据库限制: 指令优化规则需以内置表形式存储 (如 riscv-opcodes), 禁用外部数据库依赖。
- 3) 并行操作: 多线程编译时, 扩展指令优化需线程安全 (如避免全局状态冲突)。
- 4) 通讯协议: 版本更新需通过 Git 协议分发, 禁用非加密传输 (如 HTTP)。

5.2 LLVM 约束要求

5.2.1 标准符合性

- 1) RISC-V 规范

必须严格遵循 RISC-V ISA 官方标准 (如 Volume I: Unprivileged ISA 和 Volume II: Privileged ISA)。

- 2) 工具链标准

遵循 LLVM 工具链规范（如 ELF 格式、DWARF 调试信息），需兼容 GCC 工具链的 RISC-V 后端（确保交叉工具链互操作）。

5.2.2 硬件约束

- 1) 指令执行时间：扩展指令的延迟/吞吐量需在目标硬件文档中明确标注（如 `aes32csi` ≤ 3 周期）。
- 2) 内存限制：编译器自身运行需 $\leq 4\text{GB}$ 内存（处理大型项目时），生成代码需支持 $\leq 64\text{KB}$ RAM 的 MCU。
- 3) 寄存器压力：Zfinx 扩展下，浮点运算需复用整型寄存器，避免寄存器溢出。
- 4) 对齐要求：向量指令（如 P 扩展）需保证内存访问对齐（否则触发非对齐异常）。

5.2.3 技术限制

- 1) 接口限制：仅支持标准 RISC-V 调试接口（JTAG/SDT），不支持私有调试协议；必须通过 ELF 文件传递扩展指令信息，禁止自定义二进制格式。
- 2) 数据库限制：指令优化规则需以内置表形式存储（如 `riscv-opcodes`），禁用外部数据库依赖。
- 3) 并行操作：多线程编译时，扩展指令优化需线程安全（如避免全局状态冲突）。
- 4) 通讯协议：版本更新需通过 Git 协议分发，禁用非加密传输（如

HTTP)。

5.3 OpenCV 约束要求

5.3.1 标准符合性

(1) RISC-V 指令集规范

- 国际标准来源：遵循 RISC-V 国际基金会发布的 RVV v1.0 向量扩展规范。

(2) OpenCV 编程规范

- 代码规范：强制遵循 OpenCV 官方编程规范（C++11 标准），包括模块化设计和命名规则等。
- 可移植性要求：通过 Universal Intrinsics 接口跨平台适配。

5.3.2 硬件约束

(1) 目标硬件需支持 RVV 规范。

5.3.3 技术限制

(1) 跨平台开发约束：RVV Intrinsic 仅支持 GCC ≥ 12.1 或 Clang ≥ 15.0 。

六、参考资料

[1] RISC-V Unprivileged ISA Specification

<https://riscv.org/technical/specifications/>

[2] RISC-V Privileged ISA Specification

<https://riscv.org/technical/specifications/>

[3] RISC-V Toolchain Conventions

<https://github.com/riscv-non-isa/riscv-toolchain-conventions>

[4] GCC Internals Manual

<https://gcc.gnu.org/onlinedocs/gccint/>

[5] Binutils Documentation

<https://sourceware.org/binutils/docs/>

[6] GNU Coding Standards

<https://www.gnu.org/prep/standards/>

[7] 《The RISC-V Reader》（David Patterson & Andrew Waterman）

[8] RVV v1.0 官方规范文档

<https://github.com/riscv/riscv-v-spec>

[9] RVV Intrinsic 编程指南

<https://github.com/riscv/rvv-intrinsic-doc>

[10] GCC RISC-V 支持手册

<https://gcc.gnu.org/onlinedocs/gcc/RISC-V-Options.html>

[11] Clang RISC-V 向量化指南

<https://clang.llvm.org/docs/RISCVVectorSupport.html>

[12] OpenCV Universal Intrinsics 官方文档

https://docs.opencv.org/master/df/d91/group_core_hal_intrin.html

[13] OpenCV 官方编程规范

https://github.com/opencv/opencv/wiki/Coding_Style_Guide

[14] OpenCV RISC-V 移植教程

<https://github.com/opencv/opencv/wiki/OpenCV-RISC-V>

[15] QEMU RISC-V 模拟器手册

<https://www.qemu.org/docs/master/system/riscv/virt.html>